Fleet Wise

Complete System Documentation & Failure Analysis

Prepared 20 July 2026 · Live at fleet-wise-delta.vercel.app · Source: github.com/Dobgim/fleet-wise

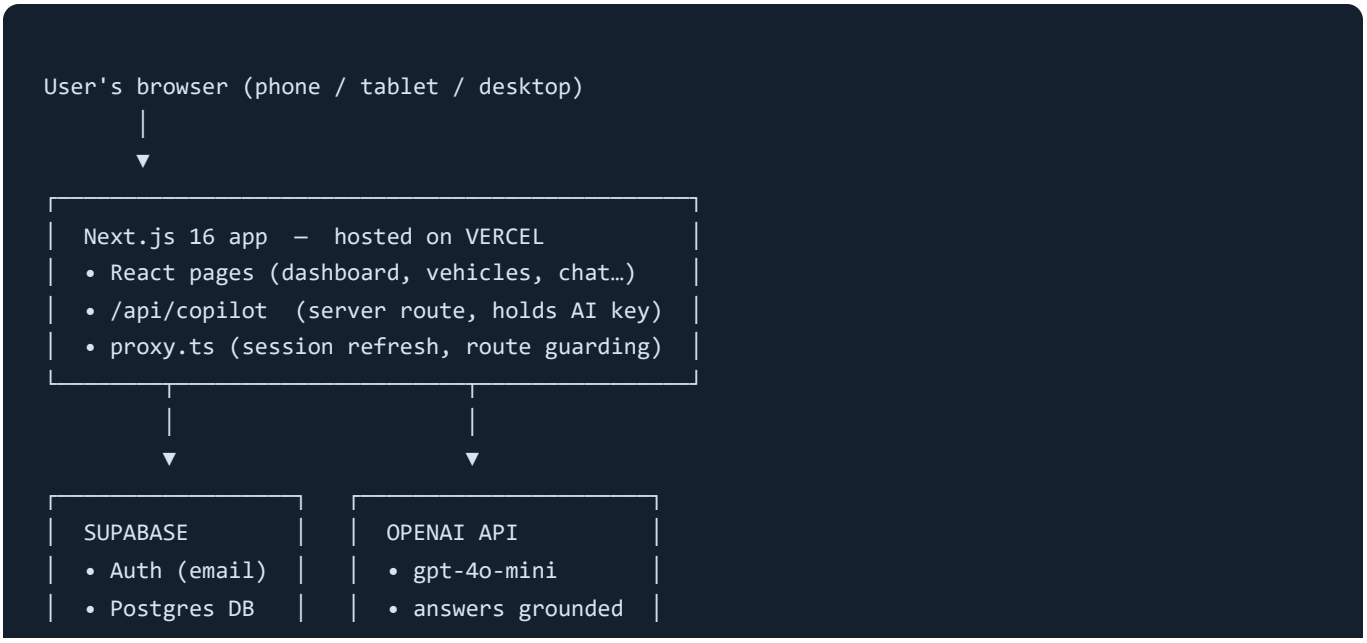
1. What Fleet Wise Is

Fleet Wise is a web application that helps anyone who owns vehicles — from a private person with two cars to a company running a fleet of trucks — stop losing money to surprise breakdowns. Users record their vehicles and every service performed (oil change, brakes, tires, battery, engine work), and the system turns that history into:

- **Maintenance alerts** — which services are overdue or coming due, based on standard service intervals (oil ≈ 6 months, brakes/tires ≈ 12 months, battery ≈ 24 months).
- **Cost intelligence** — spend per vehicle, monthly cost trends, and detection of vehicles whose repair pattern is abnormal (e.g. three brake jobs in six months), which usually signals a deeper problem or a bad repair shop.
- **An AI copilot** — a chat where the user asks questions in plain language ("Which vehicle costs me the most?", "When is the next oil change for TRK-012?") and gets answers grounded in *their own* data.

The business model is subscription-based: a free tier with limited capacity, and paid tiers (Premium $20/month, Business $100/month) that raise or remove the limits.

2. System Architecture



```
  |  • Row-Level     |   |    in fleet data     |
  |    Security      |   |    sent per request  |
  └──────────────────┘   └──────────────────────┘
```

Layer	Technology	Role
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4	All pages and UI. Mobile-first, ChatGPT-style navigation drawer, dark-mode aware.
Hosting	Vercel (Hobby tier)	Builds from GitHub `main` branch automatically on every push; serves the site globally.
Authentication	Supabase Auth	Email + password signup with email confirmation. Sessions carried in cookies, refreshed by `proxy.ts` middleware, which also redirects signed-out visitors away from app pages.
Database	Supabase Postgres	Schema created (`0001_init.sql`): organizations, memberships, vehicles, service_records, subscriptions, ai_summaries, ai_usage — all protected by Row-Level Security.
AI	OpenAI `gpt-4o-mini` via `/api/copilot`	Server-side route; API key never reaches the browser. Falls back to a built-in rules engine if the LLM is unreachable.
Email	Supabase built-in (SMTP upgrade pending)	Confirmation emails. Branded HTML template prepared; requires custom SMTP to activate.
Working data	**Browser localStorage** (see §6 — important)	Vehicles, service records, plan and AI-usage counter currently live in the browser, keyed per device.

3. The Multi-Tenant Data Model

Every row of business data belongs to an **organization** — an invisible workspace, not a legal company. A private owner's "garage" and a logistics firm's fleet are the same concept. On first sign-in the app creates the user's organization automatically (named from signup, or auto-named like "joshua's garage").

Row-Level Security (RLS) is the core safety mechanism: the database itself refuses to return or accept rows unless the signed-in user has a membership in that row's organization. Even if the application code had a bug — or someone called the database API directly with the public anon key — Postgres blocks cross-tenant access. Two helper functions (`is_org_member` , `is_org_owner`) implement the check; writes to billing and AI-metering tables are reserved for the trusted server role only.

4. The AI Copilot — How It Actually Works

The AI is **not trained** on the user's data, and no training is ever needed. It works by *grounded prompting*:

1. The user asks a question in the chat.
2. The browser sends the question *plus a compact snapshot of that user's vehicles and service history* to the app's own server route `/api/copilot`.
3. The server builds a text context (one line per vehicle and per service record), attaches a strict system prompt — *answer only from this data; never invent vehicles, dates, or costs; say plainly when data is missing* — and calls OpenAI's `gpt-4o-mini` model (fractions of a cent per question).
4. The answer returns to the chat labeled **"AI answer."** If the OpenAI call fails for any reason (no key, no credit, outage), the app computes an answer locally with its rules engine and labels it **"Offline answer (rules)."** The chat never breaks.

Because context is rebuilt on every question, the AI is always up to date with the latest records — another reason training is unnecessary.

Cost control

Each question costs real money (LLM tokens), so usage is metered per plan: Free = 10 questions/month, Premium = 200/month, Business = unlimited. When the quota is exhausted the input locks and an upgrade prompt appears. The monthly counter resets automatically.

5. Plans & Monetization

Plan	Price	Vehicles	AI questions / month
Free	$0	5	10
Premium	$20 / month	50	200
Business	$100 / month	Unlimited	Unlimited

Limits are enforced in the app today (add-vehicle blocks, chat locks). **Payment collection is not yet wired**: the Subscribe button switches plans instantly without charging, so the full user journey can be tested. Stripe integration is the designated next step; at that point the button opens a real checkout and the database `subscriptions` table becomes the source of truth, updated by Stripe webhooks.

6. Current State: What Is Real vs. What Is Simulated

Component	Status
UI — all pages, mobile & desktop	✅ Live in production
Sign up / sign in / email confirmation	✅ Live (Supabase Auth)
AI chat with OpenAI + rules fallback	✅ Deployed; LLM answers pending OpenAI billing credit
Plan limits & quota metering	⚠️ Enforced client-side only (bypassable — see §7)
Vehicles & service data	⚠️ **Stored in the browser (localStorage), not yet in Supabase.** The cloud schema exists and is applied, but the app's read/write layer still targets the browser store. Data does not follow the user across devices and is lost if browser data is cleared.
Payments (Stripe)	❌ Not built — subscribing is simulated
Branded transactional email	⚠️ Template ready; blocked on custom SMTP setup
Email maintenance reminders	❌ Not built (planned: daily scheduled job)

7. Failure Analysis — How and Why This System Will Fail

This section is deliberately blunt. Each risk lists the mechanism of failure and its mitigation.

7.1 Data loss — the biggest current risk HIGH

Mechanism: All fleet data lives in browser localStorage. A user who clears browsing data, uses private mode, switches phones, or lets iOS evict "website data" (which it does automatically for rarely-visited sites after ~7 days of inactivity) **loses every vehicle and record permanently**. There is no backup, no sync, no recovery. A paying user losing their maintenance history is a product-killing event.
Fix: Complete the Supabase data-layer migration (schema is already live) so data lives in Postgres, follows the account across devices, and benefits from Supabase's backups.

7.2 Revenue-limit bypass HIGH

Mechanism: Plan limits and the AI-question counter are enforced in JavaScript reading localStorage. Any technically-minded user can open browser DevTools, edit `fleet-copilot-local-v2`, set `plan: "business"`, and receive unlimited AI questions **billed to the owner's OpenAI account**. The /api/copilot route trusts the caller and has no server-side quota check or rate limiting.
Fix: After the data migration, meter usage server-side in the `ai_usage` table (already in the schema), verify the plan from the `subscriptions` table inside the API route, and add per-user rate limiting.

7.3 OpenAI dependency & cost exposure MEDIUM

Mechanism: (a) The account currently has no billing credit — every request returns "insufficient_quota", so users silently get rules-engine answers. (b) If credit runs out unnoticed mid-month, AI quality degrades without alerting anyone. (c) A traffic spike or abuse (7.2) can drain credit fast. (d) The API key was shared in plain text during development and should be treated as semi-exposed.
Fix: Add billing credit with a hard monthly spend cap in the OpenAI dashboard; rotate the key and set a usage alert; log fallback-mode frequency.

7.4 Large fleets will break the AI context MEDIUM

Mechanism: Every service record is sent with every question (capped at 300 vehicles / 2,000 records). A Business-tier customer with years of history will eventually exceed the model's context window or make each question needlessly expensive — the design that works for 10 vehicles fails at 1,000.
Fix: Query only relevant rows per question (the intended server-side design once data is in Postgres), summarize old history, and cache monthly summaries in `ai_summaries`.

7.5 Free-tier infrastructure limits MEDIUM

Mechanism: Supabase free projects are **paused after ~1 week of inactivity** — sign-ins then fail until manually restored. Vercel Hobby has bandwidth/function limits and forbids commercial use at scale. Gmail SMTP (planned) caps ~500 emails/day and lands in spam more often than a real domain. Growth or inattention turns any of these into an outage.
Fix: Upgrade Supabase and Vercel to paid tiers before real users arrive; move email to a proper domain + Resend.

7.6 AI answer quality LOW

Mechanism: The system prompt forbids invention, but LLMs can still miscalculate (e.g. summing costs across many records) or overstate confidence. Maintenance predictions use fixed intervals, not manufacturer schedules or actual mileage accumulation — a taxi and a weekend car get the same "oil every 6 months".
Fix: Compute numeric aggregates in code and hand them to the model pre-calculated; add per-vehicle usage-based intervals; keep the "AI answer" label so users know the source.

7.7 Operational blind spots MEDIUM

Mechanism: There is no error monitoring, no uptime alerting, no automated tests, and one developer machine (which already hit a full-disk failure during this build). Failures will be discovered by users, not by the operator.
Fix: Add Sentry (free tier) for error reporting, an uptime ping on the live URL, and a minimal test suite around quota logic and the insights engine before adding features.

7.8 Trust & brand risks LOW

Mechanism: Emails currently arrive from "Supabase Auth", which reads as phishing to cautious users; the app lives on a `vercel.app` subdomain rather than its own domain; and there are no Terms of Service or privacy policy despite storing user emails and (soon) payment status.
Fix: Custom domain (~$10/yr), custom SMTP sender, and basic legal pages before charging money.

8. Priority Roadmap (from risk to feature)

#	Action	Eliminates
1	Migrate reads/writes from localStorage to Supabase Postgres	§7.1 data loss, enables 2–3
2	Server-side plan & quota enforcement + rate limiting in /api/copilot	§7.2 revenue bypass
3	Stripe checkout + webhooks → `subscriptions` table	Simulated billing
4	OpenAI credit + spend cap + key rotation	§7.3
5	Custom domain, Resend SMTP, branded emails, legal pages	§7.5 email, §7.8
6	Scheduled daily reminder emails; relevant-rows AI context	§7.4, completes core promise

7	Sentry, uptime monitoring, tests	§7.7

End of document — Fleet Wise, 20 July 2026. Regenerate this report any time; the source HTML lives alongside the project.